06 The keystone project — build this first

> **Canadian Insurance Document RAG — public, deployed, evaluated**
>
> One repository that closes gaps 1, 2, 3, and 4 simultaneously. This is not four separate sprints; it is one project engineered to be *inspected*. Give it your next three weeks, undivided.
>
> It also doubles as the single best artifact to attach to outreach at insurance-tech and document-AI startups — the exact companies you have already been researching.

Non-negotiable requirements

- ☐ **Hybrid retrieval:** BM25 (sparse) + dense embeddings, fused. Not dense-only — dense-only is the tutorial answer and reads that way.
- ☐ **A cross-encoder reranker** over the top-k candidates. Be ready to explain, in an interview, why reranking beats simply retrieving more chunks.
- ☐ **Semantic chunking** by document boundary (clause, section, table), not by character count. Insurance policies are structured documents — exploit that.
- ☐ **An `evals/` directory.** Retrieval-only evaluation first: recall@k and MRR measured *before* any generation evaluation. Then ragas for groundedness, answer relevance, and context precision.
- ☐ **Results as a table in the README**, with a baseline row. “Dense-only recall@5: 0.61 → hybrid + rerank recall@5: 0.87” is the sentence that gets you an interview.
- ☐ **A tracing layer.** LangSmith free tier, or a custom trace log writing latency, token count, retrieved chunk IDs, and cost per query. The discipline is what is being read, not the vendor.
- ☐ **Tests.** pytest on the retrieval and chunking logic. A golden-set regression test that fails CI if retrieval quality drops.
- ☐ **Dockerfile** that actually builds, plus a docker-compose for the vector store.
- ☐ **GitHub Actions workflow** running tests and the eval suite on every push.
- ☐ **Deployed at a live URL** — Fly.io, Railway, Render, or AWS ECS. Not Vercel. Put the URL in the repo header and on your portfolio.
- ☐ **A README that opens with the eval table**, not with an architecture diagram. Lead with the number.

What to say about it. The interview story is not “I built a RAG.” It is: “Naive retrieval failed on policy exclusion clauses because they are semantically similar to the coverage clauses they contradict. I measured that failure at recall@5 of 0.61, added BM25 to catch exact clause numbers and a cross-encoder to reorder, and got to 0.87. Here is the eval harness that proves it.” That paragraph is worth more than the other seventeen projects combined.

07 The 30 / 90 / 180 day plan

Days 1–30 — build the keystone, and apply in parallel

Do not wait for the project to finish before applying. The 2027 cycle is opening right now and waiting three weeks costs you more than an incomplete portfolio does.

Week	Build track (≈15 h/wk)	Search track (≈8 h/wk)
Week 1	Corpus assembly and chunking. Get 50–100 Canadian insurance documents. Build the semantic chunker. Write the golden eval set (30–50 question/answer pairs) *before* writing retrieval.	Rewrite resume under the new-grad framing. Fix openRoles and the skills list. Apply to 10 roles.
Week 2	Dense retrieval baseline. Measure recall@k. Add BM25, fuse, measure again. Add the cross-encoder reranker, measure again. Record every number.	Apply to 12 roles. Send 5 referral requests. Post a build-in-public update on LinkedIn with the first eval numbers.
Week 3	Generation layer, ragas suite, tracing. pytest coverage on retrieval and chunking. Dockerfile, GitHub Actions, deploy to a live URL.	Apply to 12 roles. Two coffee chats. Start light DSA maintenance (3 problems/wk).
Week 4	README with the eval table up top. Demo video, 90 seconds. Add to portfolio as the featured project. Retrofit tests and a Dockerfile onto parkingTO.	Apply to 12 roles. Attach the RAG repo to every AI-role application. Re-contact anyone who went quiet.

Days 31–90 — credibility cleanup and volume

- ☐ Add pytest + Vitest/RTL to the LangGraph mail agent and one more repo. **Two repos with real tests beats twelve without.**
- ☐ Rewrite every project's `contributions` array with at least one number per project.
- ☐ Prune `skills` to provable claims only (section 5).
- ☐ Build a small **MCP server** — an insurance-document lookup tool fits naturally onto the RAG project. MCP is the fastest-moving keyword in current agentic postings and you already have adjacent LangGraph tool-calling experience.
- ☐ Sustain 12–15 targeted applications per week with a referral attempt on each.
- ☐ DSA maintenance at 3–5 problems per week for the large-company pipeline (arrays, hashing, sliding window, two pointers, graphs, DP basics).
- ☐ Do two mock interviews — one behavioural, one system design.

Days 91–180 — differentiate and convert

- ☐ Ship one open-source contribution to a library you already use (LlamaIndex, ragas, Chroma). A merged PR in an AI tooling repo is disproportionately strong signal.
- ☐ Write two technical posts: the RAG eval findings, and the fine-tuning comparison from GenDetect. Publish on your portfolio, cross-post to LinkedIn.
- ☐ Begin building the customer-facing story that makes you an FDE candidate in 2028 — your instinct to write targeted outreach to insurance-AI startups is exactly right. Keep pulling that thread.
- ☐ If nothing has landed by day 150, widen deliberately: contract work, agencies, government (Ontario Public Service hires juniors), and mid-size non-tech companies with internal engineering teams.